Pooled Priors: Ask a Language Model How Good the Model Is Before It Answers Anything

Siwei Shan[*]
NLPQuant AI Inc.

Sunny Yang[*]
University of Illinois Urbana-Champaign
guangy4@illinois.edu

Weiyan Zhao
NLPQuant AI Inc.

Abstract

Predicting how an AI system will answer a new benchmark is hardest before it has answered anything. On the competition's online draws most benchmarks had a single model, and more than four fifths of the excess error sat at the first three label budgets, where the predictor did not yet know how strong that model was. We ask a language model to estimate it. Given the model's name and settings and a handful of the benchmark's items, the deployed prompt estimates the model's accuracy with an RMSE of 0.15 on held-out training benchmarks, as informative as about six of the model's own answers. Hiding the model's identity removes almost all of this, while naming the benchmark adds nothing, so the estimate rests on knowledge of the model rather than recall of published scores. The estimate becomes a Beta prior that revealed answers update, inside a predictor that pools labels across models and calibrates per-item difficulty ratings on the fly. On online runs the prior cuts the excess error from 0.031 to 0.011 of Brier ALC, and the final submission improves the draw-adjusted ALC by 0.022 over our pooled baseline. On identical simulated draws, the prior alone improves the submitted predictor's ALC by 0.007 ± 0.001. Code, cached model outputs and a script that recomputes every table are released.

1 Introduction

The Predictive AI Evaluation Competition asks for the probability that a subject (a model under a given configuration) answers an item correctly, on benchmarks the predictor has never seen. Predictions are scored after each subject reveals 0, 1, 3, 7, 15 and 31 of its own answers, and the score is the Brier area under that learning curve (ALC). Budgets 0 to 3 carry half of the weight. At those budgets a subject's own labels say little, so the question is what else tells the predictor how strong the subject is.

Two sources can. Other subjects on the same benchmark reveal how hard it is, and the rules allow pooling their labels. When a benchmark has a single subject, nothing in the data does. We found that a language model can fill this gap. Asked how often a named model answers items like the ones shown, Claude Opus is accurate enough to stand in for several revealed answers, while cheaper priors built from the model's metadata or its accuracy on other benchmarks are no better than guessing one half.

Our contributions:

- **A subject prior from a language model** (Section 4). We measure its value in the competition's own currency, the number of revealed answers it replaces: about six on held-out training benchmarks, two to three on the online draws, where many subjects answer nearly every item and a few labels already pin them down.

[*]Equal contribution.

Technical report, Predictive AI Evaluation Competition (PAIEC) 2026.

- **A pooled predictor with no parameters fitted to the training responses** (Section 3) that combines the prior with labels from all subjects on the benchmark, an exact-item Rasch step and per-item difficulty ratings calibrated on the revealed labels.
- **An analysis of the online runs** (Section 5) that separates which subjects a run happens to draw from what the method changes. Three runs of the same final package scored 0.130 to 0.181, tracking their budget-31 levels (0.126 to 0.175), while their excess over that level stayed between 0.005 and 0.015.
- **A simulator of the protocol and cached model outputs**, released with the code, in which the submitted predictor and its ablations rerun without an API key (Section 6).

Related work. The organisers' Bayesian Linguistic Evaluator baseline also starts from a language model's estimate, but it asks for every target response separately and revises the estimate inside the prompt. We ask once per subject and benchmark for an aggregate accuracy, update it numerically with labels pooled across subjects, and calibrate item ratings by regression on the revealed labels. Language models have also been asked to forecast a benchmark score from a description of the task [9], and a model's missing scores can be filled in from its scores on other benchmarks [10]. Both predict one aggregate score per model and benchmark. We use such a forecast as the prior of an item-level predictor. Methods that predict performance from few items usually assume items calibrated on many models, through item response theory [1, 3], anchor items [4] or amortised item models [2]. On a held-out benchmark no such calibration exists. In educational measurement terms the task is a Rasch-type model [5, 6] fitted online with very few responses per examinee, as in computerised adaptive testing [7], and the subject prior is the collateral information that sets an examinee's ability prior before any responses [8]. Here the background variables are a model's name and settings, and a language model maps them to an expected score.

2 Task and scoring

For each subject–benchmark pair, the items are split 50/50 into an acquisition pool and an evaluation pool. Acquisition items stream past in random order, and an acquisition function decides whether to reveal each one, up to 31 reveals. Predictions on the evaluation pool are scored at budgets $b \in \{0, 1, 3, 7, 15, 31\}$ with weights $0.1, 0.2, 0.2, 0.2, 0.2, 0.1$, and the ALC is averaged equally over pairs. The labels a program sees are pooled across all sampled pairs, and the rules state that "the submitted program may use the attributes and acquired labels supplied through the evaluation interface to adapt during evaluation". Because the split is drawn separately for each pair, an item one subject reveals is often in another subject's evaluation pool. The program sees each item's text and features and each subject's attributes, never reference answers. Formative evaluation on Codabench is capped at 1,000 subject–item cells, about 100 items per pair.

3 Method

No parameters are fitted to the training responses. The subject prior's calibration examples and its two constants (shrinkage 0.75 and $k_0 = 4$) come from the prior-evaluation pairs, and the remaining settings (such as $k = 8$ and the ridge penalty) were chosen in simulation on the 12 simulation benchmarks, which therefore serve as development data. Every other quantity is estimated at prediction time from the labels revealed so far in the evaluation, on the target benchmark where it has enough of them and across benchmarks otherwise, as for the rating slope below.

Subject prior. We ask Claude Opus 5.5 what percentage of the benchmark's items the subject answers correctly, given up to eight sample items (each cut to 600 characters) and the subject's attributes: name, provider, release date, harness and reasoning effort. Eight solved examples from public training benchmarks, each with a model name, two sample items and that model's true accuracy, precede the question and fix the scale. The estimate q_s is shrunk toward one half, $m_s = 0.5 + 0.75\,(q_s - 0.5)$, used directly at budget 0 and as a Beta prior with $k_0 = 4$ pseudo-counts once the subject's own labels R_s arrive:

$$\hat{\mu}_s = \frac{\sum_{j \in R_s} y_{sj} + k_0\, m_s}{|R_s| + k_0}. \tag{1}$$

A worker asks first from the sample items it has seen, often just one, and once more when eight are available. It stops asking once the subject has eight labels of its own. If Opus gives no number, the predictor asks Claude Sonnet 5.5 and then Claude Haiku 5.5.

Pooling across subjects. When other subjects have at least ten labels on the benchmark, subject s's rate is shrunk toward their success rate $\bar{y}_{-s}$ with $k = 8$ pseudo-counts, $\hat{\mu}_s = (\sum_{j \in R_s} y_{sj} + k\,\bar{y}_{-s})/(|R_s| + k)$, and the two subjects' language-model estimates shift the pooled rate by half their difference. Deviations are measured against item-level predictions rather than the benchmark mean, so a correct answer on a hard item raises $\hat{\mu}_s$ more than one on an easy item.

Item level. Three terms move a prediction away from $\hat{\mu}_s$. (i) When other subjects have answered the target item itself, one Newton step on the item's logit shift under a Rasch model with prior variance $\tau^2 = 1$:

$$\operatorname{logit} \hat{p}_{sj} \leftarrow \operatorname{logit} \hat{p}_{sj} + \frac{\sum_{s' \neq s} (y_{s'j} - \hat{p}_{s'j})}{\sum_{s' \neq s} \hat{p}_{s'j}(1 - \hat{p}_{s'j}) + 1/\tau^2}. \tag{2}$$

(ii) The similarity-weighted mean residual of the 30 nearest revealed items, by cosine similarity of hashed character n-grams. (iii) A difficulty rating: Claude Opus 5.5 gives the probability that a typical current model answers the item correctly, and residuals are regressed on the standardised rating with a ridge penalty, so the rating is calibrated on the fly to the subjects at hand. Given each pair's true accuracy, Opus ratings lower the item-level Brier score on 70,563 responses from 11 simulation benchmarks from 0.1508 to 0.1387, against 0.1440 for Claude Haiku 5.5 (`src/item_signal_oracle.py`).

Acquisition. The acquisition function reveals an item only if no subject has revealed it yet on this benchmark, and always reveals when the remaining items are no more than the remaining budget. This maximises the number of evaluation items that carry an exact-item label from someone else. The rule looks only at which items were revealed, never at outcomes, so pooled rates stay unbiased.

Implementation. The platform runs predictions in several worker processes and recreates them at every budget, and workers need not share memory or disk. Every subject prior is written to several candidate cache directories and read from any of them. A failed rating is not written to disk, so other workers retry it, and a failed prior call can be retried once its lock is three minutes old. Calls retry on rate limits, and hard caps per worker (150 prior calls, 300 item ratings) bound the cost. When a cap is hit or no key is present, the predictor drops the language-model terms and runs on pooled labels alone. One formative run costs about $10 to $15.

4 How much is the prior worth?

We evaluated the prior on 87 subject–benchmark pairs from 25 public training benchmarks against each pair's true accuracy, leaving out the eight pairs that serve as calibration examples. Each estimate saw five sample items cut to 600 characters. To express the error in the competition's currency, we find the number of the subject's own labels n at which a Laplace estimate $(k + 1)/(n + 2)$ has the same mean squared error, averaged over the same pairs with $k \sim \mathrm{Binomial}(n, \text{accuracy})$.

Table 1 shows the result. Opus and Sonnet estimate accuracy with an RMSE of 0.14, each worth about seven revealed answers, and Haiku is worth three. Opus gave no number on the eight pairs of two benchmarks, where the deployed predictor falls back to Sonnet and then Haiku. With that fallback and the shrinkage chosen leave-one-benchmark-out, the deployed prior is worth six labels, and removing every benchmark that supplied a calibration example still leaves Opus worth six. Cheap priors that use no language model did not work. Rank correlations between true accuracy and priors from the model's name, its release date or its accuracy on other benchmarks ranged from -0.5 to 0.46 across benchmarks, and using the subject's accuracy on other training benchmarks as its budget-0 prediction scored 0.252, against 0.250 for a constant one half.

The value of a prior depends on the subjects. When accuracy sits near 0 or 1, a few labels already pin it down, and the same prior error is worth fewer labels. Many online subjects answered nearly every item, and there the prior was worth two to three labels (Section 5).

Table 1: Subject prior quality on held-out training pairs. "Labels" is the number of the subject's own answers a Laplace estimate needs to match the prior's mean squared error. Model rows use the 71 pairs on which all three models gave a number. Every estimate used the deployed prompt with five sample items. The deployed row uses all 79 pairs, with the shrinkage chosen leave-one-benchmark-out, and a 90% interval from a bootstrap over benchmarks.

Prior	Pairs	RMSE	Correlation	Labels
Constant 0.5	71	0.217	–	0
Claude Haiku 5.5	71	0.185	0.63	3
Claude Sonnet 5.5	71	0.142	0.76	7
Claude Opus 5.5	71	0.141	0.76	7
Deployed: Opus, then Sonnet, then Haiku	79	0.153 [0.118, 0.184]	0.69	6 [3, 13]
Opus, calibration benchmarks removed	48	0.149	0.78	6

Table 2: Online formative runs (lower is better), mean $\pm$ standard error over runs. "Adjusted" is relative to the pooled baseline after correcting each pair for its budget-31 level. "Excess" is the weighted Brier above each pair's budget-31 level. Codabench submission IDs and per-run results are in Appendix B. The subject-prior row pools four versions that differ only in deployment details and excludes the first deployment.

Version	Runs	Pairs	Raw ALC	Adjusted	Excess
Pooled baseline, no subject prior (v6)	3	26	0.162	0.000 ± 0.004	0.031 ± 0.006
+ subject prior	5	43	0.177	-0.017 ± 0.002	0.011 ± 0.002
+ Opus item ratings (final, 1017682)	3	26	0.158	-0.022 ± 0.004	0.008 ± 0.003

Estimating or remembering? A language model might recall published scores rather than estimate them. We tested this on the same 71 pairs with two variants of the deployed Opus prompt. Hiding the subject's identity (name, provider, release date and model identifier) raises the RMSE from 0.141 to 0.211, close to the constant's 0.217, so the estimate depends on knowing the model. Naming the benchmark, which the deployed prompt never does, changes it from 0.141 to 0.136 (90% benchmark-bootstrap interval for the change -0.021 to $+0.010$). A model that retrieved a published score for this model on this benchmark should gain from the name, and this one does not. The online benchmarks, from the organisers' held-out pool, point the same way: the prior's error there was 0.16 to 0.19 (Section 5). We release the estimates for all 87 training pairs (`data/llm/prior_estimates.csv`) as a starting prior for other predictors.

5 Online results

Each formative run draws nine subject–benchmark pairs (eight in some runs) from seven fixed benchmarks, and in our runs five or six pairs had a benchmark to themselves. Pooling therefore applied to few pairs, and the subject prior carried the method.

Separating the draw from the method. Raw scores mostly reflect which subjects a run draws. We use each pair's Brier score at budget 31 as its difficulty level, since by then the predictor knows the subject's accuracy well. Two quantities follow. The *adjusted* ALC subtracts, per pair, the ALC expected from its budget-31 level, a quadratic fitted on the baseline runs, so the baseline is zero by construction. The *excess* is the weighted Brier above each pair's budget-31 level. It is a diagnostic of how fast the predictor learns the subject, not a bound, and it can be slightly negative. Both use the method's own budget-31 score, which the method also affects. Three runs of the final package scored 0.181, 0.130 and 0.164, with budget-31 levels of 0.175, 0.126 and 0.149 and excess of 0.006, 0.005 and 0.015. The spread between them follows the draw.

Where the gain comes from. Table 2 gives the result. The subject prior lowers the adjusted ALC in all five of its runs, by 0.017 on average, and cuts the excess from 0.031 to 0.011. Without any adjustment, every run with the prior had a lower excess (0.004 to 0.017) than every baseline run (0.024 to 0.042). Opus item ratings add a further 0.005, within noise on three runs. Table 3 shows where. The baseline's excess at budget 0 was 0.119: with one subject and no labels it could not

Table 3: Online excess Brier at each budget, averaged over pairs (Brier at that budget minus Brier at budget 31).

Version	$b = 0$	$b = 1$	$b = 3$	$b = 7$	$b = 15$
Pooled baseline	0.119	0.048	0.023	0.018	0.007
+ subject prior	0.026	0.018	0.008	0.009	0.005
+ Opus item ratings (final)	0.038	0.025	0.003	-0.004	0.000

Table 4: The submitted predictor on identical simulated draws (90 draws). Differences are paired over draws, mean $\pm$ standard error, against the variant with both language-model terms off.

Variant	ALC	B_0	Difference	Better in
Laplace estimate on own labels	0.2020	0.250	$+0.0008 \pm 0.0002$	20% of draws
Neither term	0.2012	0.250	–	–
Item ratings only	0.1992	0.244	-0.0020 ± 0.0004	68% of draws
Subject prior only	0.1941	0.221	-0.0071 ± 0.0009	80% of draws
Both (submitted)	**0.1924**	0.216	-0.0088 ± 0.0010	83% of draws

tell a model that answers nearly everything from one that answers half. The prior lowers it to 0.026 and 0.038 in the two later versions, which run the same prior on different draws. Both lie between the baseline's excess after one and after three labels, the two-to-three-label value from Section 4. Working back from the budget-0 excess, the prior's online error was about 0.16 to 0.19, in line with the 0.15 measured offline.

Deployment. Because workers are recreated at every budget, a prior kept in memory reaches only the workers that asked for it. Our first deployment asked only after several items of a pair, which many workers never saw, and gained 0.004 instead of 0.017. Asking from the first item and sharing results on disk fixed this. For future editions, a scratch directory shared by the workers of one evaluation, or workers that persist across budgets, would let language-model submissions call each model once instead of once per worker, cutting cost and removing a source of run-to-run variation.

6 The submitted predictor in simulation

Online versions differ in their draws as well as their code. To compare components on identical draws, we ran the submitted predictor in an online-regime simulator: nine pairs over seven of the 12 simulation benchmarks per draw, mostly one subject per benchmark, 120 items per benchmark and the subject attributes as the online inputs give them, over 90 draws (three seeds of 30). Every draw starts from a fresh state, as a new evaluation does. Item ratings come from cached Opus ratings. Subject priors are real Opus calls with the deployed prompt, cached so that every variant sees the same estimates (641 calls, about $10). We then switched the subject prior and the item ratings off in turn, and added a smoothed baseline that predicts $(k + 1)/(n + 2)$ from the subject's own labels.

Table 4 shows the result. With mostly one subject per benchmark, the predictor without its language-model terms is barely better than the smoothed baseline, so nearly all of the gain comes from those terms, and the subject prior is the larger of them. With item ratings on, adding it lowers the ALC by 0.0068 ± 0.0009 and helps in 80% of draws. With the prior on, item ratings add 0.0017 ± 0.0004. The prior acts where the method predicts: Brier at budget 0 falls from 0.244 to 0.216, and the gap closes by budget 7, once the subject's own labels dominate. The simulated gain is smaller than the online 0.017 from Table 2, which compares different draws and is the less controlled of the two.

7 Pooling when subjects share a benchmark

The hidden test set may have more subjects per benchmark than the formative runs. To test the pooling machinery on its own, we simulated the protocol with per-pair 50/50 splits, streaming acquisition with the coverage rule of `labeling.py` applied to item IDs (this simulator forces a reveal against the full budget of 31 rather than the current checkpoint, which differs only near the end of a stream), nested budgets and labels pooled across pairs, with identical splits and streams for every

Table 5: Brier ALC in simulation (lower is better), averaged over pairs. v5 pools benchmark and pair rates and adds item offsets. v6 adds the exact-item step. The official baseline is the empirical mean of the subject's own labels.

Regime	Baseline	v5	v6, random reveals	v6, coverage-first
300 items, 12 benchmarks	0.2412	0.1907	0.1846	**0.1835**
Online scale, 2 subjects	0.2561	0.2196	0.2192	**0.2170**
Online scale, 5 subjects	0.2310	0.1905	0.1890	**0.1886**
Online scale, 10 subjects	0.2368	0.1893	0.1858	**0.1829**
Full pools, no rating term	0.2494	0.2051	0.2036	**0.2027**

method. These runs use the pooled predictor before the subject prior was added (v6), with item ratings from a cache of Claude Haiku ratings. We report three regimes: (i) *300 items*, 12 benchmarks with 300 sampled items each and up to 30 subjects, (ii) *online scale*, 100 items per benchmark with 2, 5 or 10 subjects, averaged over three seeds, and (iii) *full pools*, all items available for acquisition, evaluation capped at 150 items per pair and no rating term. ALC is averaged equally over pairs, as in the competition.

Table 5 shows that pooling does most of the work when subjects share a benchmark, from 0.2412 to 0.1907 on the 300-item regime. The exact-item step lowers the ALC by a further 0.006, and under random reveals it helps on all 12 benchmarks. Coverage-first acquisition lowers the ALC in 9 of 11 seed-level comparisons, by 0.0011 and 0.0031 on the two seeds of the 300-item regime. Removing the rating term raises the ALC from 0.1835 to 0.1871, with the largest gains on knowledge and code benchmarks such as `cmmlu` and `cruxeval` (Appendix A). Even with pooling, ability remains the main gap: replacing each subject's estimated rate with its true accuracy lowers the ALC from 0.1835 to 0.1709, about two thirds of it at budgets 1 and 3 (Figure 1). This oracle leaves budget 0 untouched, which the subject prior also addresses.

8 Limitations

The subject prior depends on the language model knowing the subject. It returns nothing for an unnamed subject and will help less for unfamiliar ones, and the recall test in Section 4 cannot rule out that the model recognises a benchmark from its items. Online we had three to five runs per version, so the 0.005 from Opus item ratings is within noise, and the draw adjustment uses each pair's budget-31 score, which the method itself affects. We have no direct comparison with the organisers' Bayesian Linguistic Evaluator. We tried its default configuration from the baseline repository (version of 15 October 2026) on the current public data, and it failed there: its data preparation found no eligible benchmarks in the embedding dataset, and its keyword retrieval expects an `n_trials` field that the benchmark tables no longer contain. It runs a retrieval agent of up to ten language-model turns for every prediction, of which a formative run has about 3,000 (about 500 evaluation cells at six budgets), while our predictor asks once per subject and benchmark and once per item. The method needs a paid API key, and its cost grows with the number of worker processes. All benchmarks we used have binary responses.

9 Reproducibility and disclosures

Code, the simulator, cached language-model outputs keyed by item and pair, and the logs behind every simulation number are at `https://github.com/sunnyadn/paiec-pooled-priors`. The final Codabench submission is 1017682. `python reproduce.py` prints every simulation table from the logs in seconds. With access to measurement-db, which is gated, the documented scripts rebuild the item files and the prior-evaluation pairs, `reproduce.py` then recomputes Table 1 from the cached estimates, and `reproduce.py -rerun` reruns every simulation without an API key in several hours with six parallel jobs on a laptop. The online tables are computed from the Codabench scoring logs of the submissions listed under Table 2. The submission is two files, `model.py` and `labeling.py`, and needs only the Python standard library and NumPy. Without an API key it runs without the language-model terms.

Disclosures. Training data: only public measurement-db, used for development, simulation, the eight calibration examples and the two prior constants. Pretrained models: Claude Opus 5.5 through the Anthropic API for subject priors and item ratings at test time, with Claude Sonnet 5.5 and Claude Haiku 5.5 as fallbacks. During development we also compared Claude Fable 5.1, GPT-6 Astra and GPT-6.1 Sol as raters. Development used LLM coding assistants (Claude Code and Codex).

References

[1] F. Maia Polo, L. Weber, L. Choshen, Y. Sun, G. Xu, M. Yurochkin. tinyBenchmarks: evaluating LLMs with fewer examples. arXiv:2402.14992, 2024.

[2] S. Truong, Y. Tu, P. Liang, B. Li, S. Koyejo. Reliable and efficient amortized model-based evaluation. arXiv:2503.13335, 2025.

[3] V. Hofmann, D. Heineman, I. Magnusson, K. Lo, et al. Fluid language model benchmarking. arXiv:2509.11106, 2025.

[4] R. Vivek, K. Ethayarajh, D. Yang, D. Kiela. Anchor Points: benchmarking models with much fewer examples. arXiv:2309.08638, 2023.

[5] G. Rasch. Probabilistic Models for Some Intelligence and Attainment Tests. Danish Institute for Educational Research, 1960.

[6] F. M. Lord. Applications of Item Response Theory to Practical Testing Problems. Erlbaum, 1980.

[7] W. J. van der Linden and C. A. W. Glas, editors. Elements of Adaptive Testing. Springer, 2010.

[8] R. J. Mislevy. Exploiting auxiliary information about examinees in the estimation of item parameters. Applied Psychological Measurement, 11(1):81–91, 1987.

[9] J. Park, E. Mendes, G. Stanovsky, A. Ritter. Anticipatory evaluation of language models: predicting LLM benchmark scores from descriptions. arXiv:2509.20645, 2025.

[10] Y. Zeng, D. Papailiopoulos. You don't need to run every eval. arXiv:2606.24020, 2026.

A Per-benchmark results

Table 6: ALC per benchmark on the 300-item regime. Subjects: number of subjects simulated on the benchmark.

Benchmark	Subjects	Baseline	v5	v6, coverage-first	no rating term
`agi_elo`	20	0.2244	0.1854	**0.1825**	0.1869
`cmmlu`	16	0.2615	0.1833	**0.1821**	0.2022
`ceval`	11	0.2954	0.2477	**0.2410**	0.2441
`cruxeval`	30	0.3151	0.2316	**0.2061**	0.2191
`bird_sql`	30	0.0792	0.0761	**0.0742**	0.0754
`harmbench`	29	0.2603	0.2196	0.2065	**0.2034**
`editbench`	30	0.3315	0.2401	0.2364	**0.2305**
`financebench`	16	0.2146	**0.1992**	0.2023	0.2027
`helm_afr`	22	0.3245	0.2548	**0.2495**	**0.2495**
`afrimedqa`	23	0.2442	0.1895	**0.1789**	0.1931
`agentdojo`	28	0.2206	0.1679	**0.1613**	0.1633
`benger`	24	0.1541	0.1327	0.1310	**0.1306**

B Online runs

C Variants that did not improve the submission

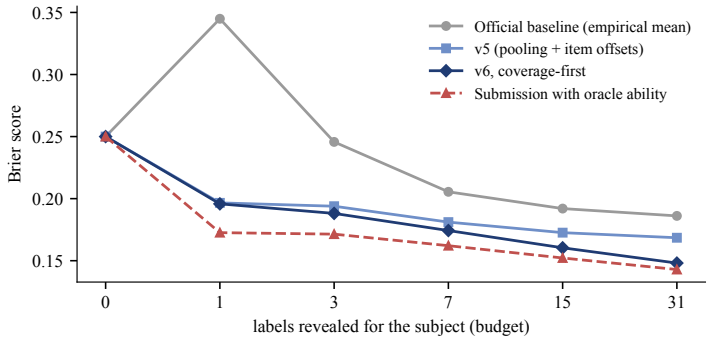


Figure 1: Brier score by budget on the 300-item regime. The oracle line replaces the estimated ability of each subject with its true accuracy on the benchmark.

Table 7: Every online formative run in Table 2. B_{31} is the mean Brier at budget 31, the draw's difficulty level.

Submission	Version	Pairs	Raw ALC	B_{31}	Excess
1016627	v6	9	0.158	0.116	0.042
1016628	v6	9	0.170	0.143	0.027
1016651	v6, no API key	8	0.158	0.134	0.024
1017498	+ subject prior (v19)	9	0.169	0.164	0.006
1017499	+ subject prior (v19)	9	0.159	0.151	0.008
1017545	+ subject prior (v23)	8	0.195	0.183	0.011
1017562	+ subject prior (v25)	8	0.179	0.169	0.010
1017629	+ subject prior (v27)	9	0.184	0.167	0.017
1017607	final	8	0.181	0.175	0.006
1017628	final	9	0.130	0.126	0.004
1017682	final (selected)	9	0.164	0.149	0.014

Table 8: Variants tried and dropped. Differences in pair-averaged Brier ALC against the predictor of the same stage (negative is better).

Variant	Result
Online Rasch model instead of shrinkage (v7, two seeds)	-0.0001 / $+0.0012$
Multi-dimensional item ratings (v8)	$+0.0010$; $+0.004$ at online scale
Staged acquisition, ability first	$+0.0021$
Acquisition stratified by predicted difficulty	$+0.0013 \pm 0.0012$
Haiku answer agreement (online)	worse on 3 runs
Text-learned difficulty (n-grams, embeddings, LoRA)	correlation ≤ 0.08
Prior ensembles (Opus twice, Opus with Sonnet)	RMSE 0.139–0.142 vs 0.141